

ML-UNG: Prefix-Frontier Entries and Predicate-Safe Block Navigation for Filtered Vector Search

Anonymous Author(s)

ABSTRACT

Label-filtered nearest-neighbor search needs both a way to enter eligible regions and a graph that connects them effectively. These choices are coupled: a simpler label graph may require more entry points, while a finer grouping can restrict geometric navigation. We present ML-UNG, a graph index that makes this tradeoff explicit. A terminal-prefix forest pairs with a covering entry frontier, replacing global minimal-superset reduction with bitmap intersection and prefix-ancestor checks. Independently constructed blocks then join exact-label groups under a shared prefix. When that prefix contains the query labels, a block permits proximity edges across its groups while preserving eligibility. Level-tagged search states use only the adjacency of their authorized scale. We establish label-plane coverage and conditional predicate safety. In an Amazon screening study at measured Recall@10 ≥ 0.90 , the prefix-base system is 2.77–18.43 $\times$ faster than the optimized LNG system on three workloads with mean selectivity below 6%. Among 14 tested configurations, the per-workload winner has a Trie base on seven of nine workloads; the LNG base wins near 10% and 30%. These results identify complementary roles for entry simplicity and coarse navigation.

1 INTRODUCTION

Vector search increasingly appears inside structured database queries. A catalog search combines semantic similarity with product categories and availability; document retrieval restricts embeddings by topic or access labels. The index must find nearby vectors among those satisfying the whole predicate. We study conjunctions of labels: a vector is eligible when its label set contains every query label. Selectivity denotes the fraction of eligible vectors.

Graph indexes must reconcile two organizations of this data: proximity in vector space and eligibility in label space. Filter-aware expansion, as in ACORN [11], improves navigation through a global graph. Label-aware systems instead expose metadata structure. UNG [2] partitions vectors by exact label set, connects groups through minimal strict-superset relations, and realizes those relations as vector edges. Curator [6] shares label-specialized virtual indexes through a clustering tree. Our starting point is UNG’s containment organization, which gives every cross-group transition a clear eligibility argument.

That organization couples two sources of work. Search first identifies the minimal observed label sets containing the query. It then traverses a vector graph whose group transitions respect containment. Entry minimality may require comparisons across unrelated label branches, while exact-group boundaries constrain which geometric transitions are available. On Amazon, 602,453 vectors occupy 482,387 exact-label groups. A query can consequently spend

substantial work both finding entries and navigating fragmented groups. The two costs need to be addressed together: reducing one can increase the other.

Design entries and connectivity together. A canonical label trie connects each observed label set to its nearest terminal descendants. Its entry set consists of the first eligible terminal on every eligible prefix branch. This *prefix frontier* covers all eligible groups in the resulting forest. It can be found without comparing eligible terminals across different branches for set minimality. The frontier may contain more groups than UNG’s minimal-superset entries, but those extra entries are what make the simpler connectivity usable. Figure 2 shows this dependency on one catalog query.

Authorize a region, then navigate geometrically. The same trie supplies a way to relax exact-group boundaries. Every vector below a prefix R_b contains R_b . If the query labels Q satisfy $Q \subseteq R_b$, a local proximity graph may connect these vectors across groups in either label direction. The prefix thus certifies a whole search region. We materialize such regions as *blocks*, whose direct members exclude separately emitted descendant blocks. Queries that cannot authorize a block still need the finer group representation. This yields an exact-group base and independently partitioned block overlays, with one overlay as the simplest instance.

ML-UNG combines these mechanisms through level-tagged search states. Each state expands the adjacency of one physical level; entry selection and block authorization determine which states enter the bounded search. This makes predicate safety independent of the chosen LNG or prefix connectivity at each level. We also batch irregular graph-construction tasks on the GPU and give a structural rule that derives overlay thresholds and depth from dataset size and existing degree budgets.

We evaluate the tradeoff by separating matched entry/topology systems from controlled upper-level topology changes. The complete Amazon factorial contains 14 configurations over nine selectivities. At measured Recall@10 ≥ 0.90 , the prefix-base system is 2.77–18.43 $\times$ faster than the optimized LNG base on three workloads near 0.5–5%, whereas LNG wins near 10% and 30%. Adding blocks changes this picture again: the best systems at 60% and 80% combine a Trie base with an LNG overlay. The detailed profiles explain how discovery savings can be offset by seed and graph work.

The contributions are:

- **A joint entry and topology design.** A terminal-prefix forest and bitmap-backed frontier provide label-plane coverage with an explicit tradeoff between minimality work and entry count.
- **Predicate-certified block navigation.** Independent prefix partitions let queries use coarser vector graphs when a root-prefix certificate holds. We specify the seeding and level-local search algorithm and establish its conditional safety properties.

- **An analysis of the entry–navigation tradeoff.** A complete topology factorial and query profiles identify where discovery savings are displaced into seed and graph work, and where blocks recover throughput. Supporting studies examine structural configuration and GPU construction.

2 PROBLEM AND MOTIVATION

2.1 Label-Constrained Nearest Neighbors

Let $\mathcal{D} = \{(x_i, A_i)\}_{i=1}^N$, with $x_i \in \mathbb{R}^d$ and finite, nonempty observed label sets A_i . Query (q, Q, K) asks for the K nearest vectors in

$$\mathcal{E}(Q) = \{i : Q \subseteq A_i\}, \quad s(Q) = |\mathcal{E}(Q)|/N. \quad (1)$$

The distance function is fixed across compared methods. For workloads with at least K eligible points, Recall@ K is the intersection of the returned and exact filtered top- K sets, divided by K . Our experiments use $K = 10$ and exact containment ground truth. Query $Q = \emptyset$ is valid and admits all vectors.

Every distinct observed label set S defines an exact group $G_S = \{i : A_i = S\}$. Let G denote the number of such groups, distinct from the vector count N . Labels are unique within a set and sorted under one fixed total order. The canonical label trie stores this sequence; its terminals represent observed groups. A group-level edge describes permitted navigation between groups. Materializing it as vector edges is a separate construction step, so group-edge counts are not vector-edge counts.

2.2 An Application Example

Consider a catalog in which a denotes English, b denotes books, and c denotes in-stock. A semantic query constrained to books has $Q = \{b\}$. Groups $\{b\}$, $\{a, b\}$, and $\{a, b, c\}$ are eligible; group $\{a, c\}$ is not. Under order $a < b < c$, the first two eligible groups lie on different prefix branches. UNG’s minimal-superset entry set contains only $\{b\}$, because set-containment edges can reach the larger groups. A prefix graph needs both $\{b\}$ and $\{a, b\}$ as entries. Removing the second entry would remove an entire eligible branch from graph-level coverage.

Conversely, a block rooted at prefix $\{a, b\}$ may safely join the $\{a, b\}$ and $\{a, b, c\}$ groups for query $\{b\}$. Query $\{b, c\}$ cannot certify that same block; it must use a finer representation containing the eligible $\{a, b, c\}$ group. This is the reason for two granularities: block size trades geometric freedom against the strength of the predicate certificate, and determines which scale a query can authorize.

2.3 Where Existing Organizations Spend Work

A global proximity graph, a label-partitioned index, a containment graph, and a filter-first scan pay for different work. Figure 1 summarizes the work each organization exposes. With a global graph, sparse eligible vertices can force extra traversal or neighborhood expansion; richer construction may alleviate the problem. With label-specialized partitions, queries must identify and combine suitable partitions. Curator’s clustering-based virtual indexes address this explicitly [6] through query-time composition. With UNG, entry discovery and fine-group traversal both remain relevant. Filter-first exact scanning has $\Theta(|\mathcal{E}(Q)|d)$ distance work

even when the fraction $s(Q)$ is small, because a large database can still yield many eligible vectors.

UNG’s LNG is the cover graph of observed strict set containment: $S \rightarrow S'$ if $S \subset S'$ and no observed label set is strictly between them. It uses a trie to discover minimal observed supersets; that lookup trie does not define its cross-group edges. Our prefix instance changes both the group relation and the matching entry contract. Our block overlays then change the navigation unit. The resulting query time combines discovery, seed setup, and graph traversal, motivating the separate measurements in Figure 1.

3 PREFIX-FRONTIER ENTRY DISCOVERY

3.1 Entry Coverage Without Minimal-Superset Reduction

Let $C_Q = \{S : Q \subseteq S, G_S \neq \emptyset\}$ be the eligible observed label sets. Write $A <_p S$ when the sorted sequence of A is a proper prefix of that of S . Our entry frontier is

$$F(Q) = \{S \in C_Q : \nexists A \in C_Q \text{ with } A <_p S\}. \quad (2)$$

UNG’s entry set instead excludes S whenever an eligible proper subset of S is observed. Every prefix is a subset, but many subsets are not prefixes. Consequently the LNG-minimal entry set is contained in $F(Q)$, and the containment can be strict. Figure 2 shows the smallest relevant distinction: $\{b\} \subset \{a, b\}$ is not a prefix relation under $a < b$, so both groups must remain in $F(\{b\})$.

Connect each terminal to its nearest terminal descendants in the canonical trie. This forms a directed forest, with one incoming edge per non-root terminal. It has exactly $G - r$ group edges, where r is the number of terminals without a terminal ancestor. Thus the count is $G - 1$ only if $r = 1$. The LNG cover graph has at most $O(G^2)$ edges, and two-level subset families can attain a quadratic count. Actual density depends on the observed label-set family.

Proposition 1: group-level frontier coverage. Every eligible observed terminal is reachable from a terminal in $F(Q)$ by a path consisting entirely of eligible terminals in the prefix forest.

Proof. For an eligible terminal S , take the first eligible terminal A on its root-to- S trie path. If no earlier eligible terminal exists, $A \in F(Q)$ by definition. Consecutive terminals from A to S are joined by the nearest-terminal edges. Their label sets all contain A , hence contain Q . This also covers $A = S$ and $Q = \emptyset$. $\square$

The proposition concerns the label plane. Lifting it to exhaustive vector reachability requires retaining every frontier group as a seed, each directed local graph being reachable from its seeded or incoming portals, and accessible materialized cross edges for each required group transition. Finite candidate capacity, approximate local graphs, pruning, and seed selection can violate those additional conditions. These additional conditions motivate the finite-budget Recall measurements.

3.2 The Measured Entry Algorithm

The Measured Trie provider stores, for each label a , a compressed bitmap I_a of groups containing a , and for each terminal S , its proper terminal ancestors $\text{Anc}_p(S)$. It intersects postings to obtain C_Q and tests only prefix ancestors to produce $F(Q)$ (Algorithm 1). This

Global proximity graph Query $\rightarrow$ filtered expansion Work: examined neighborhoods Tension: eligible connectivity	Label-aware partitions Query $\rightarrow$ partition traversal Work: filter preparation + search Tension: composition and pruning	Containment graph Query $\rightarrow$ minimal supersets Work: entry + exact-group search Tension: many fine boundaries	Filter-first scan Query $\rightarrow$ eligible IDs $\rightarrow$ scan Work: $ \mathcal{E}(Q) $ distances Tension: absolute candidate mass
---	---	--	---

Mean selectivity	OL system	Discovery (ms)	Seed setup (ms)	Graph (ms)
0.499%	LNG + optimized LNG entry	20.352	0.195	2.076
	Trie + prefix-frontier entry	0.645	1.769	0.868
9.907%	LNG + optimized LNG entry	36.631	0.646	98.747
	Trie + prefix-frontier entry	3.778	14.744	323.081

Figure 1: Top: conceptual organizations and their main sources of work. Bottom: measured per-query stage times from the separate Amazon profile at each system’s Recall crossing. Both bases use their matched entry provider, so this is a system comparison. Timings include instrumentation and concurrency effects; they are not the batch latency used to compute primary QPS.

Algorithm 1 Bitmap-backed prefix frontier (measured provider)

Require: query labels Q , label bitmaps I_a , proper terminal ancestors

```

1: if  $Q = \emptyset$  then
2:   return precomputed terminal roots of the prefix forest
3: if any query label has no posting then
4:   return  $\emptyset$ 
5: copy the smallest query-label posting into  $C_Q$ 
6: for each remaining query label  $a$  do
7:    $C_Q \leftarrow C_Q \cap I_a$ 
8:  $F \leftarrow \emptyset$ 
9: for each  $S \in C_Q$  do
10:  if no  $A \in \text{Anc}_p(S)$  belongs to  $C_Q$  then
11:    append the group identifier of  $S$  to  $F$ 
12: return  $F$ 

```

algorithm avoids comparing S against eligible terminals on unrelated branches. The bitmaps and ancestor lists are initialized before steady-state timing; per-query intersections and ancestor tests remain in the discovery time.

For comparison, both LNG-semantic providers first use a trie walk to collect a prefix-minimal candidate frontier, rather than enumerate all eligible supersets. They then reduce this frontier under set containment. The original version sorts candidates by label cardinality and removes sets containing an already retained minimum. The optimized version uses cardinality buckets when the size span is small, with a sorting fallback; it still performs pairwise sorted-set containment checks for minimality. It preserves the original entry definition. The prefix provider changes both the entry definition and the data structures used to compute it.

Cost and output tradeoff. Let $c = |C_Q|$, $f = |F(Q)|$, and A_Q be the number of proper-ancestor membership probes actually performed. If $T_\cap$ is the cost of copying and intersecting compressed

postings and t_ϵ bounds one bitmap membership probe, prefix discovery costs

$$O(|Q| + T_\cap + c + A_Q t_\epsilon + f). \quad (3)$$

For maximum label length h , $A_Q \leq ch$. Bitmap representation and compression affect $T_\cap$ and t_ϵ ; treating both as unit-time set operations would hide substantial work. The stored ancestor lists require up to $O(Gh)$ entries, in addition to postings and trie storage. The empty predicate uses the precomputed root frontier in $O(r)$ output time.

For the LNG providers, let c_{LNG} be the number of frontier candidates, T_{cand} their discovery time, and P_Q the number of pairwise minimality tests. With sorted label sets of length at most h , their cost is bounded by

$$O(T_{\text{cand}} + \text{Order}(c_{\text{LNG}}) + P_Q h), \quad P_Q \leq c_{\text{LNG}}(c_{\text{LNG}} - 1)/2, \quad (4)$$

where ordering is $O(c_{\text{LNG}} \log c_{\text{LNG}})$ for sorting or $O(c_{\text{LNG}} + \Delta)$ for cardinality buckets over span Δ . The prefix algorithm restricts which relations are tested, but may return more groups and hence increase seed distance work and graph traversal. We report these downstream costs separately.

4 PREDICATE-CERTIFIED BLOCK NAVIGATION

4.1 Two Granularities, One Vector Universe

Prefix-frontier entry changes how search starts; blocks change what it can search locally. Physical level 0 contains exact groups and their vector graphs. Physical level 1 adds vector graphs over groups aggregated into blocks. Within an authorized block, proximity edges can connect vectors from different exact groups directly, avoiding a requirement to follow the fine label relation for every transition. The base remains necessary for queries that cannot certify a useful coarse region. Figure 2 uses the catalog example to connect entry selection with this change of scale.

We call this a 1L index: one overlay above the base, hence two physical levels. A 2L index has levels 0, 1, and 2. More generally, an m -overlay index has $m + 1$ physical levels. All levels reference the

Table 1: Canonical topology notation; L means LNG and T means prefix Trie. Each upper level is an independently built block overlay.

Code	Level 0	Level 1	Level 2
$\emptyset L[T]$	Trie groups	absent	absent
$1L[T L]$	Trie groups	LNG blocks	absent
$2L[T LT]$	Trie groups	LNG blocks	Trie blocks

Algorithm 2 One independent block partition

Require: canonical trie, terminal weights $w(v)$, threshold T

- 1: **for** nodes v in postorder **do**
- 2: $u(v) \leftarrow w(v) + \sum_{c \in \text{child}(v)} u(c)$
- 3: **if** $v \neq \text{root}$ and $u(v) > T$ **then**
- 4: mark v as a block root; $u(v) \leftarrow 0$
- 5: assign each terminal to its nearest marked ancestor (including itself), if one exists
- 6: **return** roots and their direct-member lists P_b

same vector identifiers, but their block counts, represented membership, and vector-edge counts can differ. The levels aggregate label-space regions over shared vectors; the query algorithm below determines their participation.

A configuration is (m, T, τ, e, r) : overlay count, increasing thresholds, topology per physical level, entry provider, and routing policy. In particular $\tau_0 \in \{L, T\}$ is an independent choice; both base choices are evaluated. Historical plot labels $1L$ -LNG and $2L$ -LT denote the fixed-LNG-base controls $1L[L|L]$ and $2L[L|LT]$. The full notation always lists the base before the separator.

4.2 Independent Mass Partitions

Each terminal has weight equal to its group size. For threshold T_t , a bottom-up pass accumulates uncovered point mass. A non-root node becomes a block root when its accumulated mass exceeds T_t ; that mass is then removed from its parent’s uncovered total. A block owns terminals below its root up to, but excluding, another block root at the same level.

Direct-member sets are disjoint within a layer, but may form only a partial cover: residual groups below no emitted root remain represented by the base. Threshold T_t is an emission threshold, not a maximum block size. A large exact group or branching node can yield a block exceeding it. Separate partition passes for $T_1 < T_2 < \dots$ leave earlier levels unchanged; they do not imply nested direct-member sets or identical coverage at all levels.

4.3 Certification and Connectivity

Block b stores its root prefix R_b , direct members P_b , a vector entry point, and adjacency with an explicit owner level. It is authorized when $Q \subseteq R_b$. Since $R_b \subseteq A_i$ for every $i \in P_b$, the certificate implies $Q \subseteq A_i$. An unauthorized block may contain eligible points; inability to certify it says nothing about those points’ individual labels.

Algorithm 3 Terminal-prefix topology at a fixed physical level

Require: canonical trie with the level’s groups or block roots marked

- 1: push ($\text{root}, \perp$); $E \leftarrow \emptyset$
- 2: **while** stack not empty **do**
- 3: pop (v, a)
- 4: **if** v is marked **then**
- 5: **if** $a \neq \perp$ **then**
- 6: add (a, v) to E
- 7: $a \leftarrow v$
- 8: **for** each child c of v **do**
- 9: push (c, a)
- 10: **return** E

Within a block, the proximity graph need not respect exact-group boundaries. Between blocks at the same level, either a prefix relation or the LNG cover relation specifies legal transitions to more specific root-label sets. Only the layer’s direct-member points are used to realize its vector edges. Both relations preserve eligibility. Prefix edges are sparse, but are not necessarily a subset of the LNG *cover edges*: an intermediate observed set on another prefix branch can remove a cover edge that the prefix forest contains. Prefix reachability is a subset of containment reachability. This is the reason to test topology quality rather than infer it from edge counts. Algorithm 3 is used on groups at level 0 and on block roots at an overlay level.

5 QUERYING CERTIFIED BLOCKS

5.1 Seeding and Optional Routing

Before graph expansion, a query certifies block roots, applies any routing gate, and attaches physical levels to its seeds. The retained states then compete in one bounded queue. Authorization decides which regions are safe; seeding decides which of them are actually represented in that search.

Entry discovery returns actual group identifiers. The prefix instance uses Algorithm 1; LNG-base controls use one of the two minimal-superset providers. These contracts are separate from block authorization and vector entry-point setup.

We evaluate three routing rules. Let h be the highest authorized physical level at least 2, and let M_h be the sum of its authorized blocks’ direct-member counts.

- Ungated: admit the multilayer search.
- DRH-v1: admit it exactly when such an h exists.
- DRH-v2: additionally require $M_h \geq T_{\text{gate}}$.

A rejected query runs the base with the same topology and entry provider. In the two-overlay experiments, $h = 2$ whenever it exists.

For an accepted multilayer query, authorized blocks contribute their stored entry points tagged with their physical level. Each group entry receives the lowest numbered authorized overlay owning that group, or level 0 if no such overlay exists. The measured initializer prioritizes block seeds when trimming to capacity, then fills remaining space with group seeds. Retagging replaces the base seed, so the initial queue determines which physical representations participate.

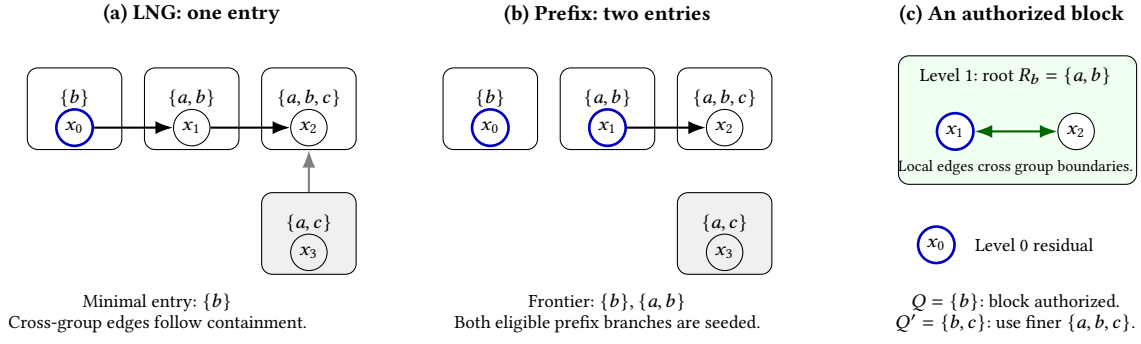


Figure 2: One query, two entry contracts, and one block overlay. For $Q = \{b\}$, blue-ringed vertices are seeds and the gray group is ineligible. Panels (a) and (b) use the same singleton groups. With threshold $T_1 = 1$, x_1, x_2 form the block in (c); its prefix authorizes either direction of local navigation for Q . Query Q' cannot authorize that block. Arrows denote permitted connections; each state uses one physical level throughout its traversal.

Algorithm 4 Level-local containment search

Require: (q, Q, K) , base and overlays H , provider e , route r , capacity L_s

- 1: **if** H has no overlays **then**
- 2: **return** BASESEARCH(q, Q, e, L_s)
- 3: $B_Q \leftarrow \{b : Q \subseteq R_b\}$
- 4: **if** r is gated **then**
- 5: find highest authorized physical level $h \geq 2$, if any
- 6: **if** no such h , or $(r = \text{DRH-v2 and } M_h < T_{\text{gate}})$ **then**
- 7: **return** BASESEARCH(q, Q, e, L_s)
- 8: discover group entries with e and assign seed levels
- 9: create authorized block seeds and group seeds
- 10: deduplicate by (i, ℓ) , mark seeds seen, and score them before trimming
- 11: prioritize block seeds; trim to L_s and initialize the queue
- 12: **while** an unexpanded state (i, δ, ℓ) remains **do**
- 13: mark the closest active unexpanded state expanded (it stays in the queue)
- 14: read only level- ℓ adjacency of i
- 15: **for** each permitted edge to j at level ℓ **do**
- 16: **if** (j, ℓ) is unseen **then**
- 17: mark it seen; offer $(j, \text{dist}(q, x_j), \ell)$ to the queue
- 18: **return** the closest at most K distinct vector identifiers in the final active queue

5.2 Level-Local Search

Search states are (i, δ, ℓ) : vector identifier, query distance, and physical level. Visited state is indexed by (i, ℓ) , while the answer set is deduplicated by vector identifier. The shared queue has a configured active capacity L_s . Algorithm 4 describes the containment path with root-prefix authorization used in the reported experiments.

Queue semantics. Candidates are ordered by (δ, i, ℓ) , breaking distance ties by vector identifier and then level. Initialization retains the closest block seeds first and fills remaining capacity with the closest group seeds. All collected seeds are scored and marked

Table 2: Illustrative ownership and seed selection, with $K = 1$ and $L_s = 2$. Stored block entries and query distances are chosen for this example.

Level	Root	Direct members	Entry (distance)
1	a	y_0, y_8	y_0 (3)
1	ab	y_1, y_2, y_3	y_1 (1)
1	ac	y_4, y_5, y_6, y_7	y_4 (2)
2	a	$y_0, \dots, y_8$	y_1 (1)

seen before this selection. Expansion marks a state without removing it from the active queue. An offered state enters if capacity is available or it precedes the current worst state; in the latter case it evicts that state, whether expanded or unexpanded. Seen marks persist after rejection or eviction. Search ends when the active queue has no unexpanded state; answers are the closest distinct vector IDs in that final queue. Thus an evicted state contributes neither a future expansion nor an answer at its evicted level.

A two-overlay execution example. Consider singleton groups with vectors $y_0, \dots, y_8$ and label sets, in order, a, ab, abc, abd, ac , followed by acd, ace, acf, ad . Concatenation denotes a set under $a < b < c < d < e < f$. For thresholds $T_1 = 1$ and $T_2 = 4$, the independent partitions produce the direct memberships in Table 2. At level 1, the a block excludes the emitted ab and ac descendants; at level 2, a owns all nine vectors. Query $Q = \{a\}$ certifies all four blocks. Its prefix-frontier group a contributes y_0 , retagged to level 1, which duplicates that block's seed and is removed during seed deduplication.

The two retained seeds are $(y_1, 1)$ and $(y_1, 2)$, distinct level states of the same vector. The first expansion uses only level-1 edges. If it exposes y_2 at distance 0.5, the queue retains $(y_2, 1)$ and $(y_1, 1)$ and evicts $(y_1, 2)$ before that coarse state expands. This trace shows how extra representations can compete for a fixed budget even with isolated adjacency. It also shows why coverage of the full frontier does not imply coverage after initialization and queue eviction.

Proposition 2: predicate safety. Assume all group seeds are eligible and every overlay-tagged group seed is a direct member of an authorized owner block; base cross-group edges preserve strict containment; block seeds are direct members of authorized blocks; and each overlay edge is expanded only with an authorized source owner matching its state level, and stays within its direct-member block or follows a same-level strict-superset block transition. Then every vector whose query distance is computed in this search is eligible.

Proof. Eligible group seeds and certified block seeds satisfy Q . An intra-group edge preserves the label set. A base cross edge moves to a superset, preserving Q . Inside a certified block every direct member satisfies Q ; a cross-block edge moves to a root containing the source root and hence Q . Induction over generated states proves the claim. $\square$

Construction and root-prefix authorization establish these invariants before traversal, allowing distance evaluation without per-neighbor predicate tests.

Proposition 3: layer isolation and fallback. A successor inherits its source state’s physical level, and adjacency owned by other levels is rejected. Induction therefore gives level-local expansion with no promotion or fallback. When the router rejects an overlay before seeding, deterministic execution with identical base entries and budget reproduces the base-search path. The rejected query additionally pays for authorization and routing.

5.3 Query Cost

Let D_q be distance evaluations, E_q inspected adjacency entries, A_q candidate insertion attempts, and P_q state expansions. Let T_{entry} include group discovery and vector-seed setup, excluding distances already counted in $D_q d$. All scored seeds, including discarded ones, contribute to D_q . Direct certification costs $T_{\text{auth}} = O(\sum_b(|Q| + |R_b|))$ plus metadata traversal. The measured default array queue can scan its active capacity even when an insertion is rejected, so a safe bound is

$$T_q = O(T_{\text{entry}} + T_{\text{auth}} + D_q d + E_q + (A_q + P_q)L_s + L_s \log L_s). \quad (5)$$

This bound exposes why fewer group edges need not lower latency: the frontier, required budget, distances, and queue operations can all change. Per-level visited state, seed buffers, and queue backing storage contribute query workspace beyond the bounded active queue. A vector reached at different levels can incur repeated distance evaluations.

6 CONSTRUCTION AND HIERARCHY CONFIGURATION

6.1 Batched Construction of Irregular Graphs

The base and overlays share two computational tasks: constructing a local vector graph for a variable-size member set, and selecting cross edges for an irregular pair of member sets. Launching a GPU operation for every tiny group repeats preparation and launch costs. Conversely, a few large blocks can dominate local graph construction. Our pipeline separates host metadata, memory-bounded GPU batches, and final host adjacency so that each cost remains visible.

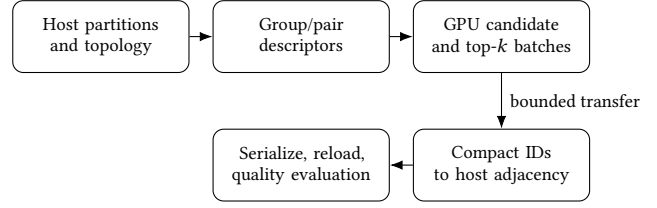


Figure 3: Construction dataflow. The GPU accelerates declared numerical stages; topology and output adjacency retain host ownership. Structural validation and query-quality evaluation are distinct checks.

Local graphs. The hybrid hierarchy builder handles small blocks with exact top- R graphs (or the configured bounded-complete alternative), medium blocks with sampled CPU Vamana, and large blocks with GPU neighbor-candidate refinement followed by diversity pruning and degree repair. The fixed thresholds are 2048 and 8192 points, with 256 sampled candidates for the middle route. The CUDA candidate path uses a FastGrnd-style adaptation of NN-Descent [3]; pruning follows relative-neighborhood principles [15]. These implementation settings are shared by compared structures. A separate full-GPU profile sends all blocks above the small completion threshold to CUDA. Neither route consults query Recall.

Cross edges. Each permitted group or block transition induces nearest-target tasks. Descriptors identify source and target member sets, dimensions, and degree budgets. A conventional GPU path computes SGEMM distance tiles and runs top- k separately; the fused path evaluates distances and selects target identifiers without materializing the complete score matrix in host memory. This uses the same distance/selection bottleneck exploited by GPU similarity search [7], specialized to many irregular tasks. Vectors and norms remain resident when they fit the configured device budget; otherwise bounded chunks are streamed. Only identifiers are transferred when downstream adjacency does not consume distances.

The base build uses the same task separation on exact groups. In the accelerated base profile, optimized host metadata, CUDA local graphs, fused cross edges, and a configured CPU additional-edge completion path jointly determine construction time. These stages contribute separately to the full-builder time.

6.2 Costs and Quality Contract

The abstract uncovered-mass partition costs $O(m|\mathcal{T}|)$ for m overlays, plus emitted membership traversal. The actual builder additionally sorts labels and member identifiers and constructs bitmaps, ancestor caches, and dense ownership maps. Prefix topology has linear edge count; the current generic overlay LNG builder tests source, target, and possible intermediate sets, giving an $O(B_\ell^3 h)$ worst-case bound for B_ℓ roots of maximum label length h . This implementation bound is different from its quadratic maximum output size.

Algorithm 5 Construction of one declared hierarchy plan

Require: vectors, label trie, plan (T_ℓ, τ_ℓ) , degree and memory budgets

- 1: **for** each level ℓ independently **do**
- 2: partition direct members and build the declared block topology
- 3: **for** each block b **do**
- 4: select its declared CPU/GPU local-graph route by member count
- 5: generate candidates, prune, and apply the degree-repair policy
- 6: form cross-edge task descriptors from the block topology
- 7: **for** each descriptor batch fitting device memory **do**
- 8: run the declared CPU or GPU distance/top- k backend
- 9: append compact identifiers to host adjacency with level owners
- 10: record effective backend, fallbacks, time, and allocated memory
- 11: serialize, reload, and validate membership and edge ownership
- 12: evaluate resulting index quality separately at the declared Recall targets

For an exact cross-edge batch with tasks (U, V) , distance arithmetic is

$$\Theta\left(d \sum_{(U,V)} |U||V|\right). \quad (6)$$

The number of selected identifiers is at most $\sum_{(U,V)} |U| \min(C, |V|)$ for target budget C . Fusing selection reduces intermediate traffic and launch overhead for this distance work. Approximate local-graph construction depends on candidate counts and refinement rounds, so we report actual work and time rather than assign it an exact-search bound.

Loaded hierarchy storage includes direct memberships and vector adjacency, plus $O(m(N + G))$ dense point/group ownership maps. Shared vector coordinates need not be duplicated per overlay. Device residency or streaming must fit the declared memory budget, including candidates and output buffers. Query workspace also includes per-level visited state, seed buffers, and queue backing storage.

6.3 A Query-Calibration-Free Instantiation

Independent overlays introduce depth and scale choices. Our degree-ratio hierarchy (DRH) uses a simple structural proxy $T + N/T$, whose continuous minimum is $\sqrt{N}$, to choose the first threshold. It then derives a growth ratio from local degree R and cross degree C :

$$T_1 = 2^{\lceil \log_2 \sqrt{N+1/2} \rceil}, \quad \rho = \max(2, \text{round}(R/C)), \quad T_{\ell+1} = \rho T_\ell. \quad (7)$$

It materializes scales while $N/T_\ell \geq C$. Ignoring the implementation depth cap and integer-overflow guard, the derived overlay count is

$$m = \max\left(0, 1 + \left\lceil \log_\rho \frac{N}{CT_1} \right\rceil\right). \quad (8)$$

Algorithm 6 DRH structure and optional mass gate

Require: vector count N , local degree R , cross degree C

- 1: $T \leftarrow \text{nearestPow2}(\sqrt{N})$; $\rho \leftarrow \max(2, \text{round}(R/C))$
- 2: **while** $N/T \geq C$ and depth cap is not reached **do**
- 3: append (T, LNG) if $N/T > R$, otherwise (T, Trie)
- 4: $T \leftarrow \rho T$
- 5: **return** overlay plan and next-scale gate $T_{\text{gate}} = T$

An overlay uses LNG when $N/T_\ell > R$ and prefix topology otherwise. The coarsest graph is thus sparsified once the estimated block count fits within one local neighborhood budget. Here N/T_ℓ is a proxy, not the measured block count; threshold partitioning leaves residuals and can emit oversized blocks. The rule balances this structural estimate; its effect on query time is evaluated empirically.

For Amazon, $N = 602,453$, $R = 64$, and $C = 4$ give the plan 1024:LNG and 16384:Trie. DRH-v1 enables the multilayer backend when an authorized physical level $h \geq 2$ exists (level 2 in the evaluated plans); DRH-v2 also requires sufficient direct mass at the highest such level, using the next unmaterialized threshold. The held-out policy experiment fixes the LNG base and derives only the overlays and gate. The policy reads no query frequencies, latency, or Recall, but its per-query authorization and mass gate naturally depend on Q .

7 EXPERIMENTAL METHODOLOGY

7.1 Questions

We ask: (Q1) When does the prefix topology with its covering frontier improve over the matched LNG system? (Q2) Which effects come from block depth and controlled topology changes? (Q3) Which profile counters explain these effects? (Q4) How well does a query-calibration-free plan transfer relative to a frozen manual candidate set? (Q5) What construction time and resources do GPU-assisted overlays require?

7.2 Execution Environment

The experiment host uses two Intel Xeon Platinum 8360Y processors (72 physical cores, 144 logical CPUs) and an NVIDIA L20 with 46,068 MiB reported device memory. Queries use 100 CPU workers. The recorded host runs CentOS 7 with NVIDIA driver 550.54.14. The artifact manifests retain the hardware snapshot, binary hashes, per-case backends, and GPU-idle checks. GPU acceleration is used for construction; query execution runs on the CPU.

7.3 Datasets and Workloads

We reuse the repository's preprocessed vector and integer-label datasets, evaluating L2 distance on the stored floating-point coordinates. Canonical tries sort stored label IDs in ascending order. The development dataset is Amazon with 602,453 vectors of dimension 768 and 482,387 observed label groups. Each Amazon workload contains 1,000 queries, uses $K = 10$, and has exact containment ground truth. The measured mean selectivities are 0.499249%, 0.903055%, 5.038117%, 9.906615%, 30.027242%, 60.047491%, 80.023699%, 95.019781%, and 99.000600%. Genome,

Table 3: Held-out datasets and query-independent DRH plans. Selectivity characterizes the frozen queries and is not an input to DRH.

Dataset	N	d	Queries	Mean sel.	DRH plan
Genome	108,077	512	3,000	3.365%	256:LNG, 4096:TRIE
Genome	108,077	512	3,000	6.292%	256:LNG, 4096:TRIE
Reviews	288,065	384	3,000	0.200%	512:LNG, 8192:TRIE
Reviews	288,065	384	2,864	4.115%	512:LNG, 8192:TRIE
VariousImg	758,935	512	3,000	10.252%	1024:LNG, 16384:TRIE

Reviews, and VariousImg are held out for the automatic-policy study; their exact workload directories and fingerprints are recorded in the artifact manifest. The 99% Amazon workload contains 697 empty predicates and 303 singleton predicates $\{1\}$. The empty-predicate path is validated separately before timing.

The 0.5%, 1%, and 10% batches reuse existing predicate files; their mean query-label counts are 5.939, 5.763, and 3.396. The 5%, 30%, and 60% batches raise the mean coverage of source batches by replacing selected predicates with the frequent singleton $\{1\}$. The 80% and 95% batches use the same construction and query vectors from a 75% source batch. Label 1 covers 96.702% of the database, so the 99% batch mixes it with the empty predicate. These are batch-mean selectivities: for example, individual queries in the 5% batch range from 0.0033% to 96.702%. Predicate composition and selectivity vary together, so the sweep characterizes these workload families.

The five manual plans perturb the automatic two-overlay plan $(t, 16t)$: one LNG overlay at t ; two LNG overlays at $(t, 16t)$; two Trie overlays at $(t, 16t)$; and LNG/Trie overlays at $(t/2, 8t)$ or $(2t, 32t)$. Here t is 256 for Genome, 512 for Reviews, and 1024 for VariousImg. All share the LNG base and the presence gate used by DRH-v1; consequently the one-overlay alternative always falls back to the base under this gate.

7.4 Methods and Fairness

The principal 0L comparison is LNG topology with optimized-LNG entry discovery versus Trie topology with Trie entry discovery. The topology factorial fixes $T_1 = 1024$, $T_2 = 16384$, and ungated routing while enumerating both base topologies and every LNG/Trie upper-layer assignment through 2L. Within either base, replacing one upper letter is a controlled topology comparison because its entry provider is unchanged. Across bases we instead report a matched-provider system comparison, not a pure edge-topology effect. Routed controls are evaluated separately because routing changes the executed search algorithm. For held-out automatic-policy evaluation, however, the exact gate is fixed for DRH and every manual-alternative hierarchy; the ungated DRH variant is reported separately.

We call the 0L LNG configuration *Plain* or *0L-LNG*. It preserves UNG’s exact-label groups, LNG cover relation, and unified vector graph, with our optimized, semantically equivalent minimal-superset provider and the common refactored execution path. Within a fixed LNG base and provider, this is the control for adding overlays.

Within each comparison, query methods use the same data, query vectors, labels, exact ground truth, K , thread count, immutable search binary, graph backend, and Recall protocol. Each case executes one cold and two warm repeats of the same frozen query file in the same order. We discard the cold repeat and derive QPS from the median warm batch time. The operating point is the smallest executed L_{search} for which both warm repeats reach $\text{Recall}@10 \geq 0.90$. A separate instrumented pass collects stage times and work counters. If a method has no screen crossing, its crossing pass extends to the largest L_{search} measured for any method on that workload (or an explicitly declared common cap); only failure at that shared budget is reported as no crossing. The workload-specific search caps appear in Table 9.

7.5 Metrics

Table 8 in Appendix A defines all stage times and counters. Primary throughput is $\text{QPS} = 1000B/\text{batch latency (ms)}$. Batch size B is workload-specific: 1,000 on Amazon and 2,864 or 3,000 on the held-out workloads (Table 3). Primary query metrics are warm-median batch latency and QPS. With only two warm repeats, we report the observed throughputs without query-performance confidence intervals.

Build timing uses process wall time through files on disk, followed by structural acceptance checks; kernel time is never substituted for builder wall time. The current construction evidence contains two measured repeats per hierarchy backend, after a cold run, and a separate memory-resource pass. The composed total adds independently measured base and sidecar stage medians. We report observed dispersion across the two stage repeats. GPU cases require three consecutive idle-device snapshots before timing. Backend counters and reload checks validate execution and structure. These construction times compare the declared builders; query-quality equivalence of their output graphs has not been measured.

The exact CPU/GPU routes are fixed before measurement; Appendix D lists the base and hierarchy profiles. They differ in local graph construction, metadata preparation, and cross-edge generation, so their total-time ratios measure complete builder profiles.

8 RESULTS

The sweep compares selective-predicate batches and mixtures containing a frequent singleton, using the batch-mean selectivity defined in Section 7.

8.1 Matched Base and Overlay Choices

The best organization changes across the Amazon workloads: prefix-base systems lead near 0.5–5%, while the LNG base leads near 10% and 30%. Table 4 compares the best measured configuration within each depth and across the full 14-configuration grid.

Complete base-by-overlay factorial. The workload-wise oracle selects $2L[T|LT]$ at 0.5%, 1%, 5%, and 95%; $0L[L]$ at 10% and 30%; $1L[T|L]$ at 60% and 80%; and $2L[T|TT]$ at 99%. Thus no static topology dominates every workload. Among configurations crossing the Recall target at all nine selectivities, $2L[T|LT]$ ranks first by geometric mean of QPS normalized to the per-workload oracle (0.868), but its worst fraction of the oracle is 0.482, at 30%. The matched-provider base-only comparison is equally non-monotone: $0L[T]$ is 6.03 $\times$, 2.77 $\times$, and 18.43 $\times$ faster at 0.5%, 1%, and 5%, respectively, but is 0.60 $\times$ at 10%, does not cross at 30% ($R_{\max} = 0.8882$), and is 0.62 $\times$ –0.33 $\times$ at 60%–95% (0.56 $\times$ at 99%). These are end-to-end system ratios because the two bases use different compatible entry providers.

The controlled upper-level comparisons reveal a more stable role for Trie. With a Trie base and LNG at level 1, changing only level 2 from LNG to Trie ($2L[T|LL] \rightarrow 2L[T|LT]$) improves QPS in eight of nine workloads, by up to 1.251 $\times$; the remaining 80% point is effectively tied at 0.998 $\times$. In contrast, changing only the finer level 1 from LNG to Trie with level 2 fixed to LNG wins in five of nine workloads but loses 40.7%–52.9% at 10%, 60%, 80%, and 95%. The evidence therefore favors Trie as a sparse coarse overlay more consistently than as the fine navigation topology. The LNG base is best on these Amazon workloads near 10% and 30%; the evaluated policy does not select between base topologies.

8.2 Where Discovery Savings Go

The profiles expose the tradeoff behind the base comparison (Figure 1). At 0.499%, Trie reduces discovery from 20.352 to 0.645 ms/query, while seed setup rises from 0.195 to 1.769 ms. Graph time also falls, from 2.076 to 0.868 ms. At 9.907%, discovery still falls, from 36.631 to 3.778 ms, but seed setup rises to 14.744 ms and graph time increases from 98.747 to 323.081 ms. Cheaper entry discovery therefore coincides with both a winning and a losing end-to-end configuration. The changed frontier and topology shift work into different stages.

Blocks can recover throughput on batches with larger mean selectivity. Keeping the LNG base and its optimized provider fixed, the ungated one-overlay system reaches 27.99 $\times$ Plain and the two-overlay system reaches 27.57 $\times$ on the measured higher-mean-selectivity batches (Table 5). Both fail to cross at 30% within the shared budget. In the full grid, the best one-overlay system already leads at 60% and 80%; a second overlay is useful at 95%, where the best QPS rises from 520.35 to 564.63. Thus additional scale can help, but the base, overlay topology, and seed competition jointly determine its value.

8.3 Automatic Policy Transfer

DRH-v1 remains near Plain on Genome and Reviews (0.954–1.014 $\times$), but falls to 0.205 $\times$ on VariousImg (Table 6). Its ratios

to the best of the five manual alternatives are 0.980–1.008 on Genome/Reviews and 0.218 on VariousImg. A fixed structural plan therefore transfers unevenly. The best manual candidate on VariousImg is the one-overlay plan, whose presence gate executes base search; its advantage there comes from rejecting the overlays. The per-workload manual winner is selected in hindsight. Table 15 also compares one fixed plan per dataset, aggregated over its workloads.

The separate mass-gate experiment tests this failure mode. On VariousImg, DRH-v2 raises QPS from 547.01 to 884.18, reaching 0.336 $\times$ its same-cohort Plain. On the 4.115% Reviews workload it rejects a useful overlay and falls to 0.972 $\times$ Plain (Table 16). Increasing the admission requirement reduces one regression but loses a beneficial case.

The profiles show how routing changes work averaged over all queries in each broader workload (Table 17). On VariousImg, the multilayer backend is active for 37.4% of queries; mean scanned edges rise from 119,045.5 to 251,144.0 and graph time rises from 35.098 to 172.065 ms/query. On Reviews, activation is 18.2%, distances fall from 8,239.3 to 7,312.0, and graph time changes from 6.756 to 6.504 ms/query. These contrasting outcomes motivate an admission model that accounts for expected navigation work as well as certifiable mass. All four datasets derive two overlays, so this experiment examines scale and topology transfer at a fixed resulting depth.

8.4 Construction Cost

On Amazon, the fastest hybrid builder reduces hierarchy construction from 1000.64 to 90.49 seconds, an 11.06 $\times$ ratio; the full-GPU route takes 105.90 seconds (Table 7). Adding GPU cross-edge construction to the hybrid route increases time to 132.94 seconds, and the TF32-WMMMA route takes 1011.13 seconds. More GPU execution is therefore not uniformly beneficial for these irregular tasks.

Including the separately measured accelerated base stage gives composed times of 141.27 seconds for the fastest hybrid and 156.69 seconds for full GPU, compared with 190.38 seconds for the original CPU base builder (Table 19). The resulting ratios are 1.35 $\times$ and 1.21 $\times$. The hybrid resource pass records 4880.7 MiB peak host RSS and 1598.0 MiB peak device allocation. Table 18 reports the CPU sidecars used to enable the held-out query study.

9 DISCUSSION AND RELATED WORK

9.1 Design Implications

The entry frontier, geometric connectivity, and search budget interact. A prefix forest saves minimality work but creates more entry branches; blocks reconnect groups only where their prefixes certify a useful region. The observed winners therefore vary across the Amazon workloads. The construction rule offers a reproducible starting plan, while the VariousImg result shows that authorized mass alone can admit expensive searches. A planner that also models frontier size and expected navigation work is a natural next step.

Table 4: Best measured configuration at each Amazon batch-mean selectivity. Each depth column is optimized only within that depth; the overall column ranges over all 14 configurations. Values are QPS at Recall@10 $\geq$ 0.90.

Mean sel.	Overall		Best 0L		Best 1L		Best 2L	
	Config.	QPS	Config.	QPS	Config.	QPS	Config.	QPS
0.5%	2L[T LT]	21707.89	0L[T]	14921.90	1L[T T]	17937.69	2L[T LT]	21707.89
0.9%	2L[T LT]	14278.50	0L[T]	11591.42	1L[T L]	12447.15	2L[T LT]	14278.50
5.0%	2L[T LT]	11718.35	0L[T]	10044.23	1L[T T]	10937.27	2L[T LT]	11718.35
9.9%	0L[L]	450.17	0L[L]	450.17	1L[T L]	289.73	2L[T LT]	275.64
30.0%	0L[L]	52.77	0L[L]	52.77	1L[T L]	27.66	2L[T TT]	27.14
60.0%	1L[T L]	1613.68	0L[L]	4.07	1L[T L]	1613.68	2L[T LT]	1598.42
80.0%	1L[T L]	648.35	0L[L]	1.80	1L[T L]	648.35	2L[T LL]	647.95
95.0%	2L[T LT]	564.63	0L[L]	1.50	1L[T L]	520.35	2L[T LT]	564.63
99.0%	2L[T TT]	267.67	0L[L]	0.79	1L[T T]	264.44	2L[T TT]	267.67

Table 5: Amazon equal-Recall comparison. Plain QPS is the raw throughput of 0L[L]; every other numeric column is QPS(method)/QPS(Plain) at that method’s smallest measured point whose two warm repeats both reach Recall@10 $\geq$ 0.90. NC means no crossing in the shared measured budget.

Mean sel.	Plain QPS	0L[T]/plain	1L[L L]/plain	2L[L LT] ungated/plain	2L[L LT] DRH-v1/plain
0.499%	2476.35	6.03×	NC	NC	0.86×
0.903%	4191.04	2.77×	NC	NC	0.99×
5.038%	544.93	18.43×	NC	NC	0.85×
9.907%	450.17	0.60×	NC	NC	0.64×
30.027%	52.77	NC	NC	NC	NC
60.047%	4.07	0.62×	13.89×	15.09×	14.40×
80.024%	1.80	0.33×	20.33×	27.57×	24.22×
95.020%	1.50	0.33×	27.99×	26.46×	26.70×
99.001%	0.79	0.56×	19.57×	21.34×	18.78×

Table 6: Screen-level 2L[L|LT]/DRH-v1 results at Recall@10 $\geq$ 0.90.

Dataset	Mean sel.	Plain QPS	DRH-v1/plain	DRH-v1/best manual	Best manual plan / executed route
Genome	3.365%	153.65	0.954×	1.008	256,4096:trie,trie
Genome	6.292%	85.45	0.982×	1.007	256:lng (base fallback)
Reviews	0.200%	19319.56	1.014×	0.997	1024,16384:lng,trie
Reviews	4.115%	354.78	1.004×	0.980	1024,16384:lng,trie
VariousImg	10.252%	2657.04	0.205×	0.218	1024:lng (base fallback)

9.2 Filtered and Range-Constrained Graph Search

HNSW [10] and DiskANN [5] organize vector proximity for bounded ANN search. Filtered-DiskANN [4] incorporates labels

during graph construction. ACORN [11] supports predicate-agnostic graph search with expanded neighborhoods and compressed construction; its design includes a configurable low-selectivity pre-filtering fallback. Its operating regime therefore depends on both the graph and the fallback configuration.

Table 7: Repeated Amazon hierarchy-sidecar construction measurements. Resource values come from a separate profiling run and do not enter timing medians.

Backend	Repeats	Wall median (s)	CV	Speedup/CPU	Peak RSS (MiB)	Peak GPU (MiB)
cpu	2	1000.64	0.005	1.00×	5001.5	0.0
full_gpu	2	105.90	0.007	9.45×	4178.1	2286.0
full_gpu_wmma	2	1011.13	0.000	0.99×	4223.5	2362.0
hybrid_gpu_intra	2	90.49	0.008	11.06×	4880.7	1598.0
hybrid_gpu_intra_inter	2	132.94	0.006	7.53×	4255.6	2286.0

UNG [2] is the immediate structural baseline. Its exact-label groups, minimal-superset LNG, and vector realizations jointly preserve containment-directed navigation. We expose a different entry/topology contract: a terminal-prefix forest with its covering frontier. The forest has fewer permitted relations than general containment, but its edges are not necessarily an edge-subset of the LNG cover graph. We then aggregate compatible groups into independently authorized blocks. HNSW instead samples vector vertices across geometric layers; our scales change the label-space navigation unit.

SeRF [24], iRangeGraph [19], and UNIFY [9] compress, compose, or combine interval-specialized graphs. Dynamic range systems extend such structures to arrival and update workloads [13, 22]. Their ordered scalar intervals inspire explicit granularity, but arbitrary label containment forms a partial order and requires a different authorization argument.

Curator [6] shares label-specialized virtual indexes through a clustering tree. Its temporary indexes for conjunctions can be assembled by qualified-ID ordering and partition lookup without distance computation; its reported two-label AND results are close to pre-indexed predicates. Our blocks instead have fixed canonical prefix membership and use a root-prefix certificate to authorize traversal. FAVOR [14] selects a filter-first brute-force path below an estimated selectivity threshold and an exclusion-distance HNSW path otherwise. Its empirically chosen threshold and reported dataset-dependent crossover motivate studying multiple regimes; the absolute candidate mass remains relevant to the cost of its scan route.

Elastic Index Selection [20] and SIEVE [8] choose and share collections of indexes. Their selection problem differs from choosing block scales inside one containment-oriented system. Recent LSSG [17], an arXiv preprint, stratifies edges by label-set similarity and traverses multiple tiers. Our blocks instead aggregate direct members under a common prefix; exact root authorization permits local geometric navigation across their groups. Each search state remains at one physical adjacency level. Query-aware Routing [18], also a preprint, uses an offline performance table and a learned Recall model; DRH deliberately uses neither, accepting the resulting loss of adaptability.

9.3 Graph Construction

NN-Descent [3] and relative-neighborhood pruning [15] provide building blocks for proximity graphs. Database work improves construction with tree/graph hybrids, LSH-assisted insertion,

monotonic neighborhoods, and revised pruning [1, 12, 21, 23]. Starling [16] co-designs navigation and storage for disk I/O. GPU similarity search [7] emphasizes distance and selection throughput. Our implementation applies batching to the irregular local-graph and cross-edge tasks induced by the metadata hierarchy, with final adjacency assembled on the host.

10 CONCLUSION

Label-filtered graph search must choose both where to start and how to navigate after entry. ML-UNG couples a prefix forest with its covering frontier, then uses prefix certificates to authorize coarser block graphs. This gives a concrete division of work: the frontier covers eligible label branches, while a block relaxes exact-group boundaries for geometric search. The measured topology interactions show why neither entry count nor graph sparsity alone predicts performance. The GPU study reduces measured construction time, while policy transfer exposes the limits of size-based configuration. Together, the results favor choosing entry and navigation structures jointly at a common Recall target.

REFERENCES

- [1] Ilias Azizi, Karima Echihabi, and Themis Palpanas. 2023. ELPIS: Graph-Based Similarity Search for Scalable Data Science. *Proceedings of the VLDB Endowment* 16, 6 (2023), 1548–1559. <https://doi.org/10.14778/3583140.3583166>
- [2] Yuzheng Cai, Jiayang Shi, Yizhuo Chen, and Weiguo Zheng. 2024. Navigating Labels and Vectors: A Unified Approach to Filtered Approximate Nearest Neighbor Search. *Proceedings of the ACM on Management of Data* 2, 6 (2024), 1–27. <https://doi.org/10.1145/3698822>
- [3] Wei Dong, Moses Charikar, and Kai Li. 2011. Efficient k -Nearest Neighbor Graph Construction for Generic Similarity Measures. In *Proceedings of the 20th International Conference on World Wide Web*. Association for Computing Machinery, New York, NY, USA, 577–586. <https://doi.org/10.1145/1963405.1963487>
- [4] Siddharth Gollapudi, Neel Karia, Varun Sivashankar, Ravishankar Krishnaswamy, Nikit Begwani, Swapnil Raz, Yiyong Lin, Yin Zhang, Neelam Mahapatro, Premkumar Srinivasan, Amit Singh, and Harsha Vardhan Simhadri. 2023. Filtered-DiskANN: Graph Algorithms for Approximate Nearest Neighbor Search with Filters. In *Proceedings of the ACM Web Conference 2023*. Association for Computing Machinery, New York, NY, USA, 3406–3416. <https://doi.org/10.1145/3543507.3583552>
- [5] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnaswamy, and Rohan Kadekodi. 2019. DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node. In *Advances in Neural Information Processing Systems*, Vol. 32. Curran Associates, Inc., Red Hook, NY, USA, 11 pages.
- [6] Yicheng Jin, Yongji Wu, Wenjun Hu, Bruce M. Maggs, Jun Yang, Xiao Zhang, and Danyang Zhuo. 2026. Curator: Efficient Vector Search with Low-Selectivity Filters. *Proceedings of the ACM on Management of Data* 4, 1 (2026), 1–27. <https://doi.org/10.1145/3786635>
- [7] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2021), 535–547. <https://doi.org/10.1109/TBDATA.2019.2921572>

- [8] Zhaoheng Li, Silu Huang, Wei Ding, Yongjoo Park, and Jianjun Chen. 2025. SIEVE: Effective Filtered Vector Search with Collection of Indexes. *Proceedings of the VLDB Endowment* 18, 11 (2025), 4723–4736. <https://doi.org/10.14778/3749646.3749725>
- [9] Anqi Liang, Pengcheng Zhang, Bin Yao, Zhongpu Chen, Yitong Song, and Guangxu Cheng. 2024. UNIFY: Unified Index for Range Filtered Approximate Nearest Neighbors Search. *Proceedings of the VLDB Endowment* 18, 4 (2024), 1118–1130. <https://doi.org/10.14778/3717755.3717770>
- [10] Yu. A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2020), 824–836. <https://doi.org/10.1109/TPAMI.2018.2889473>
- [11] Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. 2024. ACORN: Performant and Predicate-Agnostic Search Over Vector Embeddings and Structured Data. *Proceedings of the ACM on Management of Data* 2, 3 (2024), 1–27. <https://doi.org/10.1145/3654923>
- [12] Yun Feng Peng, Byron Koon Kau Choi, Tsz Nam Chan, Jianye Yang, and Jianliang Xu. 2023. Efficient Approximate Nearest Neighbor Search in Multi-Dimensional Databases. *Proceedings of the ACM on Management of Data* 1, 1 (2023), 1–27. <https://doi.org/10.1145/3588908>
- [13] Zhencan Peng, Miao Qiao, Wenchao Zhou, Feifei Li, and Dongjie Deng. 2025. Dynamic Range-Filtering Approximate Nearest Neighbor Search. *Proceedings of the VLDB Endowment* 18, 10 (2025), 3256–3268. <https://doi.org/10.14778/3748191.3748193>
- [14] Junjie Song, Yu Liu, Guoyu Hu, Zhongle Xie, Ming Yang, Beng Chin Ooi, and Ke Zhou. 2026. FAVOR: Efficient Filter-Agnostic Vector ANNS Based on Selectivity-Aware Exclusion Distances. *Proceedings of the ACM on Management of Data* 4, 3 (2026), 1–25. <https://doi.org/10.1145/3802060>
- [15] Godfried T. Toussaint. 1980. The Relative Neighbourhood Graph of a Finite Planar Set. *Pattern Recognition* 12, 4 (1980), 261–268. [https://doi.org/10.1016/0031-3203\(80\)90066-7](https://doi.org/10.1016/0031-3203(80)90066-7)
- [16] Mengzhao Wang, Weizhi Xu, Xiaomeng Yi, Songlin Wu, Zhangyang Peng, Xianguyu Ke, Yunjun Gao, Xiaoliang Xu, Rentong Guo, and Charles Xie. 2024. Starling: An I/O-Efficient Disk-Resident Graph Index Framework for High-Dimensional Vector Similarity Search on Data Segment. *Proceedings of the ACM on Management of Data* 2, 1 (2024), 1–27. <https://doi.org/10.1145/3639269>
- [17] Ziqi Wang, Jingzhe Zhang, Shuo Shen, and Wei Hu. 2026. Fast Label-Filtering Approximate Nearest Neighbor Search via Progressive Label Set Stratification. arXiv:2609.15058 [cs.DB] <https://arxiv.org/abs/2609.15058>
- [18] Qianqian Xiong and Mengxuan Zhang. 2026. Query-aware Routing for Filtered Approximate Nearest Neighbors Search. arXiv:2606.19898 [cs.DB] <https://arxiv.org/abs/2606.19898>
- [19] Yuexuan Xu, Jianyang Gao, Yutong Gou, Cheng Long, and Christian S. Jensen. 2024. iRangeGraph: Improvising Range-Dedicated Graphs for Range-Filtering Nearest Neighbor Search. *Proceedings of the ACM on Management of Data* 2, 6 (2024), 1–26. <https://doi.org/10.1145/3698814>
- [20] Mingyu Yang, Wenxuan Xia, Wentao Li, Raymond Chi-Wing Wong, and Wei Wang. 2025. Elastic Index Selection for Label-Hybrid AKNN Search. *Proceedings of the VLDB Endowment* 19, 4 (2025), 617–630. <https://doi.org/10.14778/3785297.3785304>
- [21] Shuo Yang, Jiadong Xie, Yingfan Liu, Jeffrey Xu Yu, Xiyue Gao, Qianru Wang, Yanguo Peng, and Jiangtao Cui. 2025. Revisiting the Index Construction of Proximity Graph-Based Approximate Nearest Neighbor Search. *Proceedings of the VLDB Endowment* 18, 6 (2025), 1825–1838. <https://doi.org/10.14778/3725688.3725709>
- [22] Fangyuan Zhang, Mengxu Jiang, Guanhao Hou, Jieming Shi, Hua Tao Fan, Wenchao Zhou, Feifei Li, and Sibao Wang. 2025. Efficient Dynamic Indexing for Range Filtered Approximate Nearest Neighbor Search. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–26. <https://doi.org/10.1145/3725401>
- [23] Xi Zhao, Yao Tian, Kai Huang, Bolong Zheng, and Xiaofang Zhou. 2023. Towards Efficient Index Construction and Approximate Nearest Neighbor Search in High-Dimensional Spaces. *Proceedings of the VLDB Endowment* 16, 8 (2023), 1979–1991. <https://doi.org/10.14778/3594512.3594527>
- [24] Chaoji Zuo, Miao Qiao, Wenchao Zhou, Feifei Li, and Dongjie Deng. 2024. SeRF: Segment Graph for Range-Filtering Approximate Nearest Neighbor Search. *Proceedings of the ACM on Management of Data* 2, 1 (2024), 1–26. <https://doi.org/10.1145/3639324>

Table 8: Measurement definitions. A “warm median” uses the two warm repeats. Per-query stage times and work counters are first averaged over all queries in one repeat and then summarized across warm repeats.

Reported quantity	Definition
Mean selectivity	$B^{-1} \sum_{j=1}^B \mathcal{E}(Q_j) /N$ for the frozen query batch; it describes the workload and is not an input to DRH.
Recall@10	For one repeat, the mean over queries of $ R_{10}(q_j, Q_j) \cap R_{10}^*(q_j, Q_j) /10$. A crossing requires every warm repeat at that executed point to reach 0.90.
L_{search}	Capacity of the bounded candidate queue. An equal-Recall operating point is the smallest executed L_{search} that crosses the Recall target; values between executed points are not interpolated.
Batch latency / QPS	Wall time to complete all B queries with 100 query threads, after the cold repeat is discarded; $QPS = 1000B/\text{batch latency (ms)}$.
A/B speedup	QPS_A/QPS_B , using each method’s selected equal-Recall crossing. A ratio above one favors A .
$NC / R_{\max}$	NC means no Recall crossing within the workload-wide measured L_{search} budget. $R_{\max}$ is the largest minimum-over-warm-repeats batch Recall observed in that budget.
ELS / entry / authorization	Mean milliseconds per query for Entry Label Set (entry-group) discovery, vector entry-point setup, and exact block-authorization plus routing work, respectively.
Graph / residual	Mean milliseconds per query inside bounded graph traversal, and the remaining measured query time outside the named stages.
Encounters / edges / distances	The CSV counter Visited is reported as Encounters. These are mean per-query counts of first encounters through adjacency, scanned adjacency entries, and vector-distance evaluations. A first encounter is counted before distance scoring and queue admission; rejected candidates therefore count. Initial seed scoring contributes to distances, but not to this encounter counter.

Table 9: Measured Amazon crossing and screen caps.

Mean sel.	Plain crossing L	Screen cap L
0.499%	2,000	2,000
0.903%	2,200	2,200
5.038%	3,200	5,000
9.907%	26,000	64,000
30.027%	120,000	150,000
60.047%	260,000	320,000
80.024%	320,000	602,453
95.020%	320,000	602,453
99.001%	400,000	602,453

A MEASUREMENT DEFINITIONS AND SEARCH BUDGETS

Evidence provenance. Full paths and SHA256 digests for 27 validated inputs are stored in the deadline artifact manifest; their ordered aggregate digest begins 7e05bfbde4d08b7b.

B COMPLETE TOPOLOGY AND MEASUREMENT TABLES

Table 10: Zero- and one-overlay configurations in the complete Amazon base-by-overlay topology screen at Recall@10 $\geq$ 0.90. A code $mL[B|U]$ lists base topology B before the separator and upper topologies U from fine to coarse. L is LNG and T is Trie. The base uses its matched entry provider: optimized-LNG for $B = L$ and Trie for $B = T$. NC(r) gives maximum minimum-warm-repeat Recall when there is no crossing in the shared measured budget.

Mean sel.	0L[L]	0L[T]	1L[L L]	1L[L T]	1L[T L]	1L[T T]
0.499%	2476.35	14921.90	NC (0.888)	NC (0.888)	15177.24	17937.69
0.903%	4191.04	11591.42	NC (0.857)	NC (0.846)	12447.15	12144.74
5.038%	544.93	10044.23	NC (0.893)	NC (0.883)	9105.22	10937.27
9.907%	450.17	270.63	NC (0.874)	NC (0.848)	289.73	146.19
30.027%	52.77	NC (0.888)	NC (0.836)	NC (0.828)	27.66	10.50
60.047%	4.07	2.51	56.52	56.70	1613.68	839.32
80.024%	1.80	0.59	36.65	37.08	648.35	285.42
95.020%	1.50	0.49	42.06	36.82	520.35	265.13
99.001%	0.79	0.44	15.40	14.34	253.23	264.44

Table 11: Two-overlay configurations in the complete Amazon base-by-overlay topology screen at Recall@10 $\geq$ 0.90. A code $mL[B|U]$ lists base topology B before the separator and upper topologies U from fine to coarse. L is LNG and T is Trie. The base uses its matched entry provider: optimized-LNG for $B = L$ and Trie for $B = T$. NC(r) gives maximum minimum-warm-repeat Recall when there is no crossing in the shared measured budget.

Mean sel.	2L[L LL]	2L[L LT]	2L[L TL]	2L[L TT]	2L[T LL]	2L[T LT]	2L[T TL]	2L[T TT]
0.499%	NC (0.888)	NC (0.888)	NC (0.888)	NC (0.888)	19158.52	21707.89	20593.10	17565.42
0.903%	NC (0.857)	NC (0.857)	NC (0.846)	NC (0.846)	13242.16	14278.50	14062.30	12353.41
5.038%	NC (0.893)	NC (0.893)	NC (0.884)	NC (0.884)	9569.96	11718.35	11082.17	10046.26
9.907%	NC (0.848)	NC (0.849)	NC (0.822)	NC (0.823)	270.57	275.64	135.16	139.26
30.027%	NC (0.839)	NC (0.839)	NC (0.833)	NC (0.832)	24.92	25.42	27.05	27.14
60.047%	56.37	61.40	57.00	56.42	1565.04	1598.42	849.99	844.65
80.024%	35.44	49.70	35.88	35.94	647.95	646.83	305.37	313.61
95.020%	39.61	39.75	36.99	36.68	451.32	564.63	267.67	269.48
99.001%	14.69	16.79	14.41	14.28	253.95	257.77	265.85	267.67

B.1 Amazon Stage and Work Breakdown

This profile-only pass reuses each method’s measured performance operating point. Rows marked max use the best measured point because Recall@10 did not cross 0.90; they explain work but are not equal-Recall speed comparisons. Timeout denotes a declared profile timeout at the fixed 3,300-second per-case cap; no partial counters are used.

Table 12: 1L overlay-topology ablation at fixed $T_1 = 1024$, level-0 LNG, and optimized-LNG entry. QPS is reported only at a measured Recall@10 ≥ 0.90 crossing; $R_{\max}$ is the largest minimum warm-repeat Recall in the shared measured budget.

Mean sel.	1L[L L] QPS	1L[L T] QPS	Trie/LNG	LNG $R_{\max}$	Trie $R_{\max}$
0.499%	NC	NC	—	0.8878	0.8878
0.903%	NC	NC	—	0.8567	0.8464
5.038%	NC	NC	—	0.8928	0.8835
9.907%	NC	NC	—	0.8742	0.8477
30.027%	NC	NC	—	0.8359	0.8283
60.047%	56.52	56.70	1.003×	0.9511	0.9503
80.024%	36.65	37.08	1.012×	0.9346	0.9309
95.020%	42.06	36.82	0.875×	0.9289	0.9259
99.001%	15.40	14.34	0.931×	0.9325	0.9301

Table 13: 2L topology ablation at fixed $T_1 = 1024$, $T_2 = 16384$, level-0 LNG, optimized-LNG entry, and ungated routing. In 2L[L|XY], X is the level-1 topology and Y the level-2 topology. QPS appears only at a measured Recall@10 ≥ 0.90 crossing.

Mean sel.	2L[L LL] QPS	2L[L LT] QPS	2L[L TL] QPS	2L[L TT] QPS	LL $R_{\max}$	LT $R_{\max}$	TL $R_{\max}$	TT $R_{\max}$
0.499%	NC	NC	NC	NC	0.8878	0.8878	0.8878	0.8878
0.903%	NC	NC	NC	NC	0.8567	0.8567	0.8464	0.8464
5.038%	NC	NC	NC	NC	0.8929	0.8928	0.8836	0.8836
9.907%	NC	NC	NC	NC	0.8484	0.8492	0.8220	0.8229
30.027%	NC	NC	NC	NC	0.8385	0.8393	0.8333	0.8324
60.047%	56.37	61.40	57.00	56.42	0.9529	0.9538	0.9523	0.9525
80.024%	35.44	49.70	35.88	35.94	0.9357	0.9362	0.9341	0.9313
95.020%	39.61	39.75	36.99	36.68	0.9306	0.9310	0.9294	0.9271
99.001%	14.69	16.79	14.41	14.28	0.9338	0.9346	0.9325	0.9313

Table 14: Amazon detailed profile. Times are ms/query; counters are all-query means. Encounters are first visits through adjacency.

Mean sel.	Op.	Method	ELS	Entry	Auth.	Graph	Encounters	Edges	Distances
0.499%	cross	0L[L]	20.352	0.195	0.000	2.076	1966.8	23458.9	2150.2
0.499%	cross	0L[T]	0.645	1.769	0.000	0.868	988.6	11047.0	2764.4
0.499%	max	1L[L L]	19.665	0.216	0.015	4.418	1860.2	23296.2	2043.9
0.499%	max	1L[L T]	16.188	0.224	0.012	4.578	1859.2	23263.7	2042.9
0.499%	cross	2L[L LT]/DRH-v1	20.455	0.194	0.003	2.109	1968.1	23464.7	2151.5
9.907%	cross	0L[L]	36.631	0.646	0.000	98.747	25024.2	189382.5	25402.9
9.907%	cross	0L[T]	3.778	14.744	0.000	323.081	38926.2	351908.0	44682.9
9.907%	max	1L[L L]	33.451	0.645	0.017	644.477	34782.5	821915.0	35184.9
9.907%	max	1L[L T]	32.969	0.576	0.015	587.339	32344.1	757831.0	32746.5
9.907%	cross	2L[L LT]/DRH-v1	31.648	0.685	0.023	301.608	28532.4	449218.0	28936.2
30.027%	cross	0L[L]	251.380	3.963	0.000	1376.910	91953.6	580600.5	93654.0
30.027%	max	0L[T]	8.381	30.629	0.000	2420.260	88466.5	497828.0	98769.0
30.027%	max	1L[L L]	220.485	1.887	0.017	1765.145	61473.4	1112780.0	63220.3
30.027%	max	1L[L T]	226.654	1.963	0.015	1662.880	58664.4	1021480.0	60411.4
30.027%	max	2L[L LT]/DRH-v1	234.946	2.091	0.013	1755.115	64374.7	978653.5	66124.7
60.047%	cross	0L[L]	2123.720	33.187	0.000	21357.550	254657.0	1988540.0	265484.5
60.047%	cross	0L[T]	10.670	63.353	0.000	39255.750	261406.5	1453445.0	278174.0
60.047%	cross	1L[L L]	1415.925	8.882	0.017	200.452	45762.3	218706.0	56600.6
60.047%	cross	1L[L T]	1465.900	8.881	0.013	192.047	44045.4	200138.0	54883.6
60.047%	cross	2L[L LT]/DRH-v1	1408.655	9.064	0.017	195.087	45607.8	215753.5	56454.1
95.020%	cross	0L[L]	4918.875	61.074	0.000	89240.100	451225.0	3967260.0	467715.0
95.020%	timeout	0L[T]	–	–	–	–	–	–	–
95.020%	cross	1L[L L]	2307.095	12.576	0.018	113.150	44157.5	170659.0	60663.6
95.020%	cross	1L[L T]	2313.385	13.444	0.014	299.972	67643.4	309628.0	84149.5
95.020%	cross	2L[L LT]/DRH-v1	2317.090	12.571	0.024	113.679	43995.0	170239.0	60513.8

Table 15: One fixed DRH-v1 plan versus one fixed manual plan per held-out dataset (geometric-mean QPS).

Dataset	Workloads	DRH-v1 QPS	DRH-v1/manual
Genome	2	110.86	1.013
Reviews	2	2642.45	0.989
VariousImg	1	543.74	0.218

C POLICY AND CONSTRUCTION DETAILS

Table 16: Same-binary DRH-v2 screen results. Plain is measured in this routing cohort; it need not match the earlier DRH-v1 cohort.

Dataset	Mean sel.	Plain QPS	DRH-v1 QPS	DRH-v2 QPS	v2/plain
Genome	3.365%	143.73	144.50	142.57	0.992×
Genome	6.292%	82.74	83.73	83.16	1.005×
Reviews	0.200%	18210.79	17445.16	18335.39	1.007×
Reviews	4.115%	361.63	368.07	351.49	0.972×
VariousImg	10.252%	2633.06	547.01	884.18	0.336×

Table 17: Detailed profile at each held-out dataset’s broader workload (ms/query and mean work per query). Active is the fraction of all queries that execute the multilayer backend, summarized over warm repeats.

Dataset	Method	Graph ms	Encounters	Edges	Distances	Active
Genome	0L[L] (Plain)	0.124	123.8	154.3	4538.9	0.0%
Genome	2L[L LT]/DRH-v1	0.199	185.1	591.8	4601.4	1.8%
Reviews	0L[L] (Plain)	6.756	4415.8	15126.2	8239.3	0.0%
Reviews	2L[L LT]/DRH-v1	6.504	3500.1	25605.4	7312.0	18.2%
VariousImg	0L[L] (Plain)	35.098	18656.0	119045.5	18673.1	0.0%
VariousImg	2L[L LT]/DRH-v1	172.065	27356.1	251144.0	27393.7	37.4%

Table 18: Automatic DRH CPU sidecar construction.

Dataset	Wall s	Blocks	Upper	Edges
Genome	44.2	264	18	10,088,461
Reviews	120.4	217	16	27,590,479
VariousImg	633.2	190	10	80,034,774

Table 19: Composed end-to-end construction: accelerated base stage plus hierarchy sidecar versus the original CPU base builder. Composed time is the sum of independently measured stage medians.

Hierarchy backend	Stage repeats	Original CPU (s)	Composed (s)	Speedup
cpu	2	190.38	1051.42	0.18×
full_gpu	2	190.38	156.69	1.21×
full_gpu_wmma	2	190.38	1061.91	0.18×
hybrid_gpu_intra	2	190.38	141.27	1.35×
hybrid_gpu_intra_inter	2	190.38	183.72	1.04×

Table 20: Frozen base-graph construction profiles. “Optimized metadata” denotes bucketed minimal-superset discovery, optimized LNG construction, epoch-based descendants, and direct descendant coverage.

Profile	Intra-group graph	Group metadata	Vector cross-group edges
original_cpu	CPU Vamana	Original sort/LNG, hash-BFS descendants, topological coverage	Original CPU builder
current_cpu	CPU Vamana	Optimized metadata	CPU Vamana cross-edge builder
naive_gpu	CPU Vamana	Optimized metadata	Batched GPU SGEMM plus separate top- <i>k</i>
paper_fused	CPU Vamana	Optimized metadata	Descriptor-grouped fused distance/top- <i>k</i> with identifier-only writeback
accelerated_gpu	FastGrndd CUDA path	Optimized metadata	Fused GPU cross edges plus CPU exact-scan additional edges

Table 21: Frozen hierarchy-sidecar construction profiles. CPU completion is retained below the declared small-block threshold in every profile.

Profile	Intra-block route	Inter-block route
cpu	Small exact/complete, medium sampled Vamana, large CPU Vamana	CPU
hybrid_gpu_intra	Small exact/complete, medium sampled Vamana, large FastGrndd-style CUDA	CPU
hybrid_gpu_intra_inter	Same hybrid intra route	Fused CUDA top- <i>k</i>
full_gpu	Bounded completion, then CUDA	Fused CUDA-core top- <i>k</i>
full_gpu_wmma	Bounded completion, then CUDA	Fused TF32-WMMA top- <i>k</i>

D CONSTRUCTION PROFILE DETAILS

D.1 Frozen Construction Profiles

E COMPLETE RECALL-THROUGHPUT CURVES

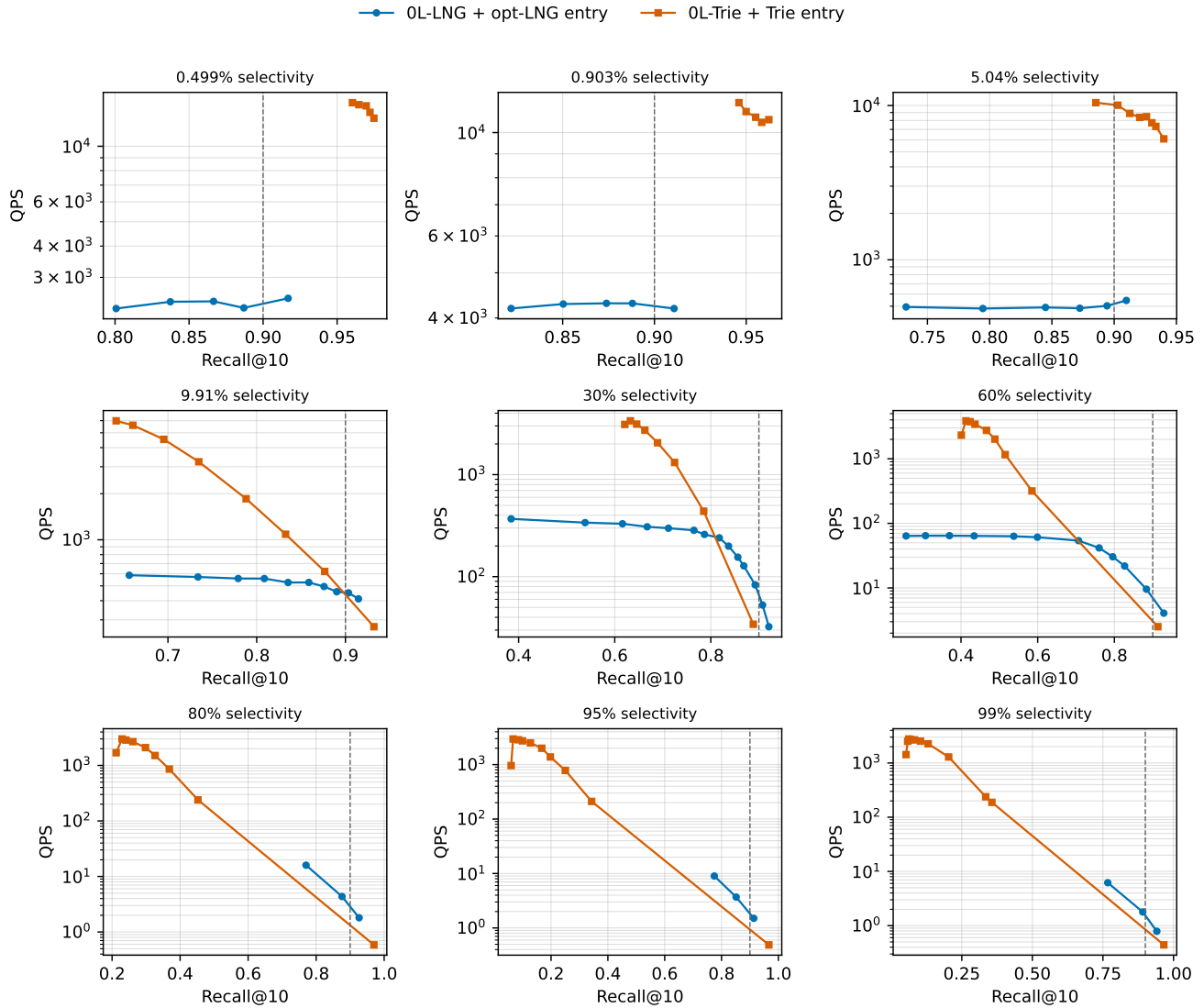


Figure 4: Measured Recall@10-QPS curves for the 0L[L]/0L[T] comparison. Markers are executed points; curves are not interpolated.

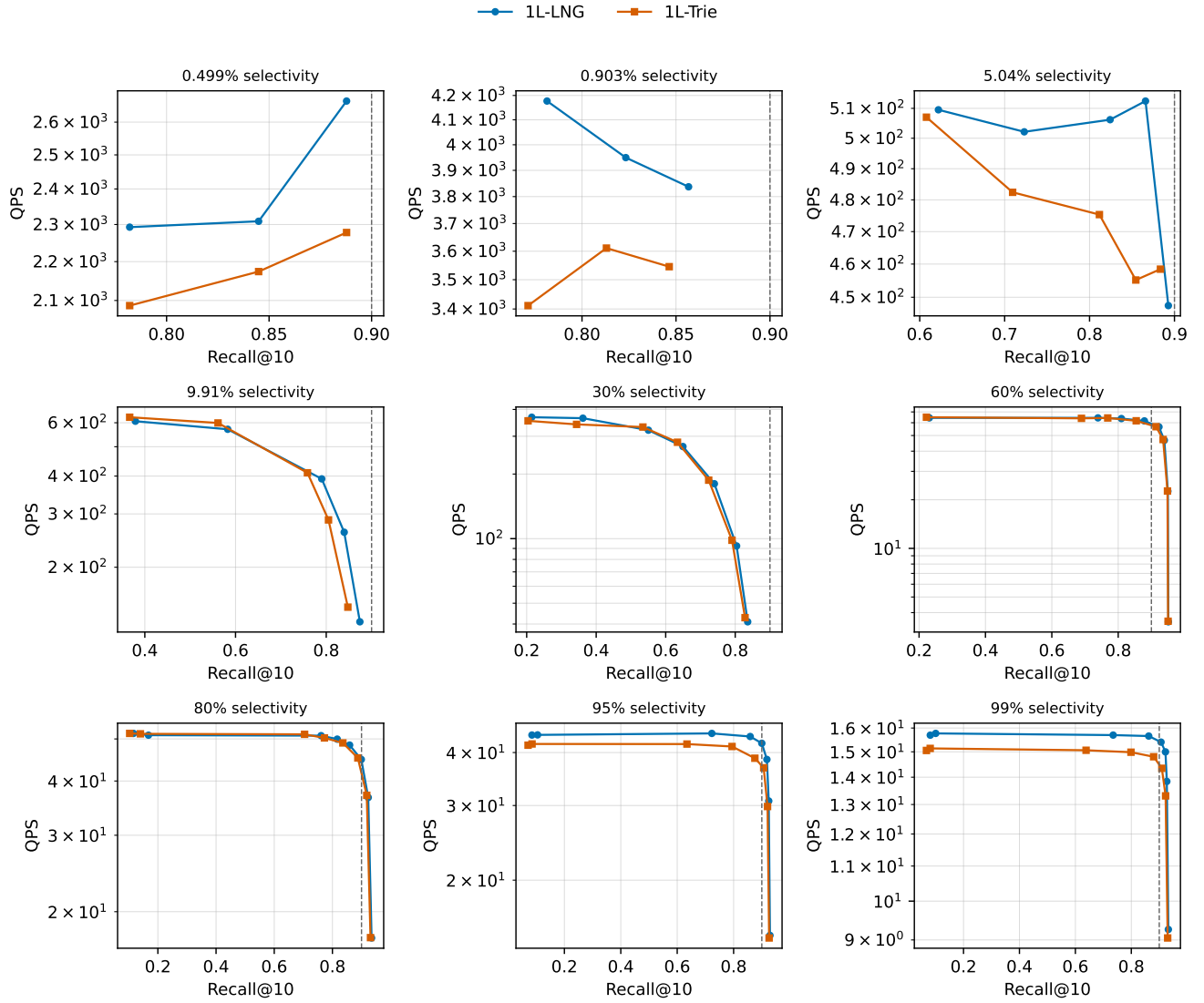


Figure 5: Measured Recall@10-QPS curves for 1L[L]/1L[T] overlay topology at fixed threshold, level-0 LNG, and entry provider. Markers are executed points; curves are not interpolated.

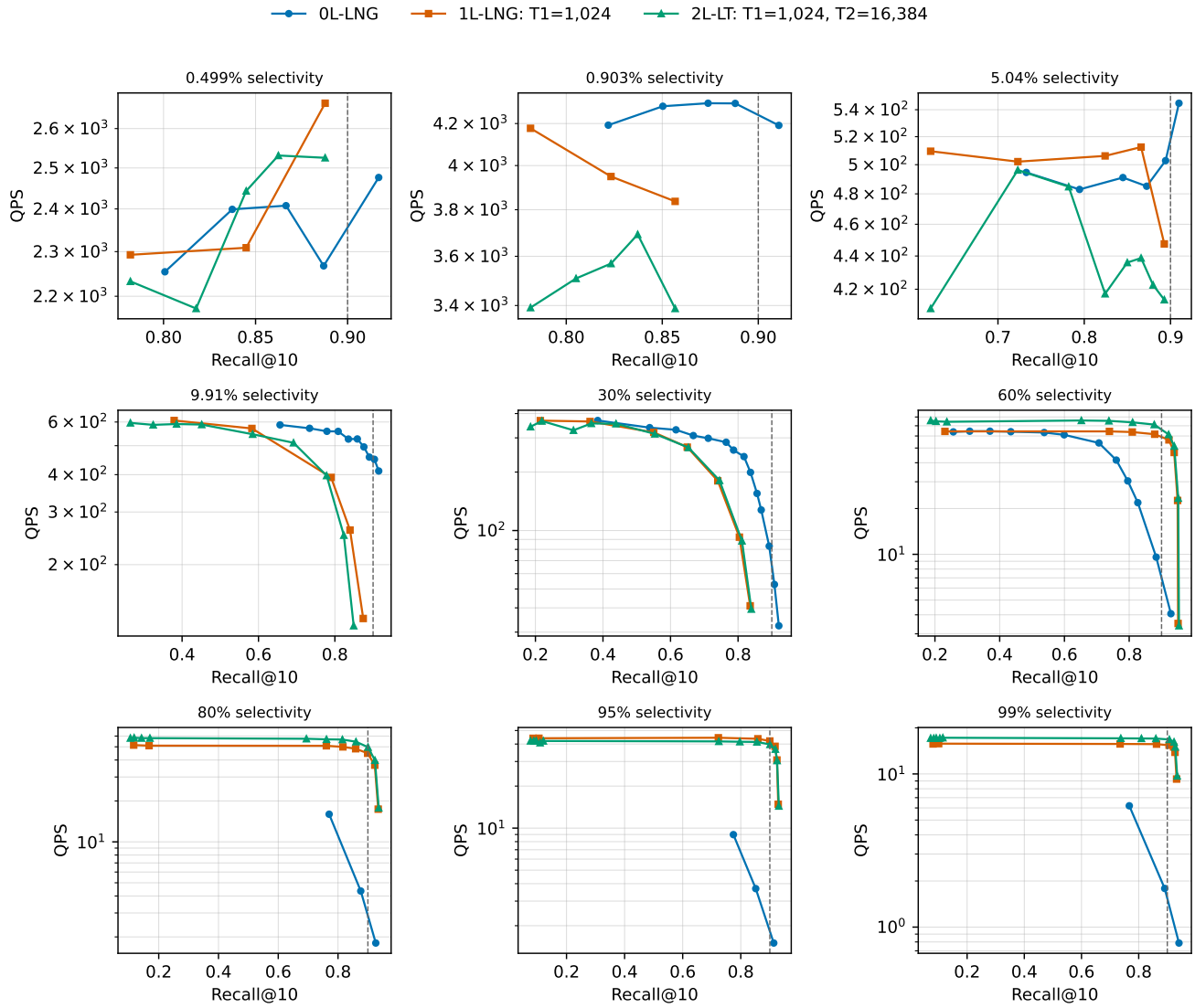


Figure 6: Measured Recall@10-QPS curves for the representative 0L/1L/2L comparison. Markers are executed points; curves are not interpolated.

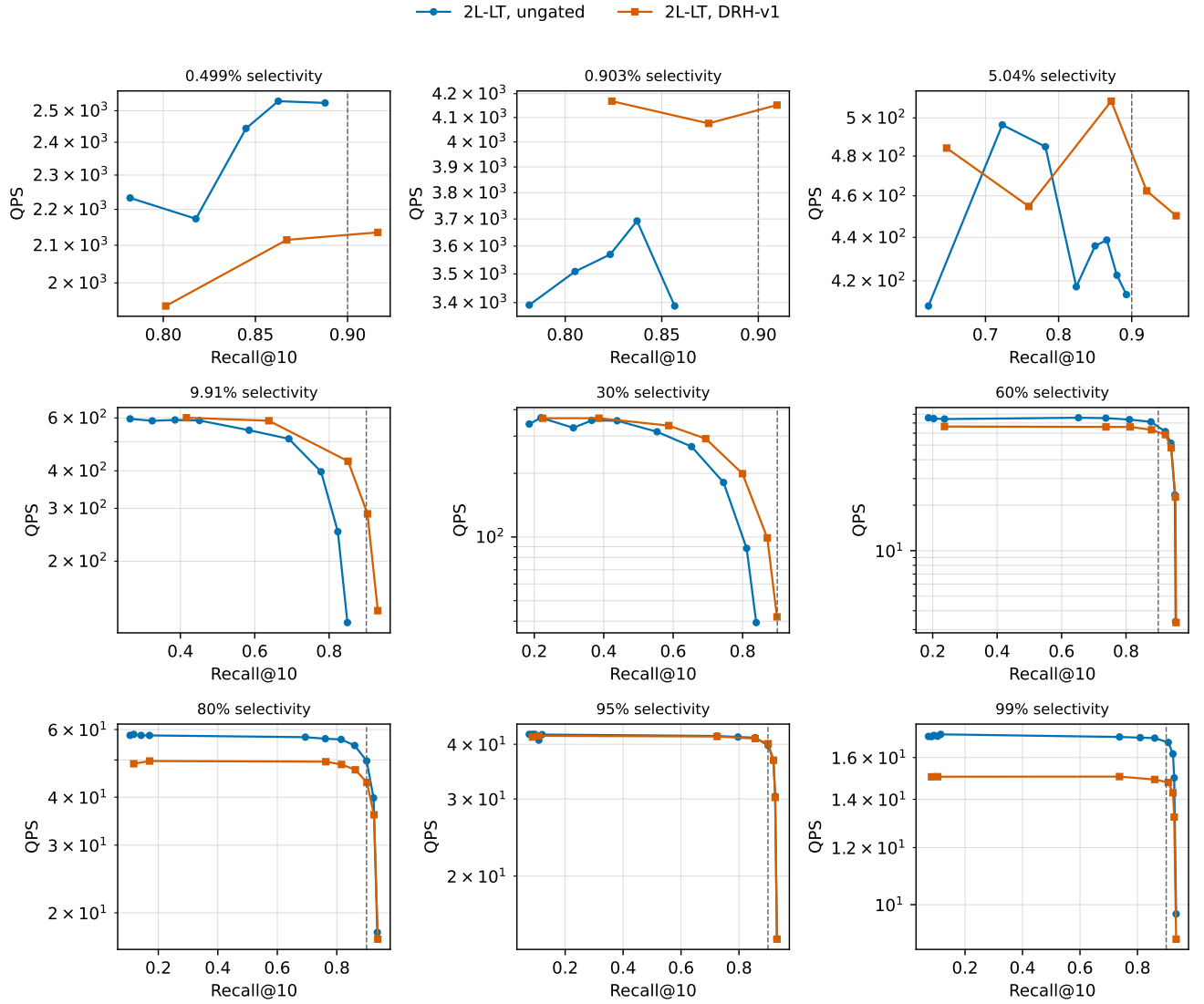


Figure 7: Measured Recall@10-QPS curves for ungated versus DRH-v1 routing for 2L[L|LT]. Markers are executed points; curves are not interpolated.